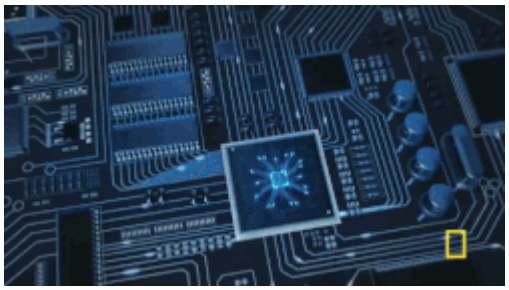
# Standardschaltwerke

| Parameter | Kursinformationen |
|---|---|
| **Veranstaltung:** | `Digitale Systeme / Eingebettete Systeme` |
| **Semester:** | `Wintersemester 2026/27` |
| **Hochschule:** | `Technische Universität Freiberg` |
| **Inhalte:** | `Register, Schieberegister, Zähler und deren Anwendungen als Rechner-Basiskomponenten` |
| **Link auf GitHub:** | https://github.com/TUBAF-IfI-LiaScript/VL_EingebetteteSysteme/blob/master/08_StandardSchaltwerke.md |
| **Autoren:** | Sebastian Zug & André Dietrich & Fabian Bär & Claude Code (Opus 5.5) |



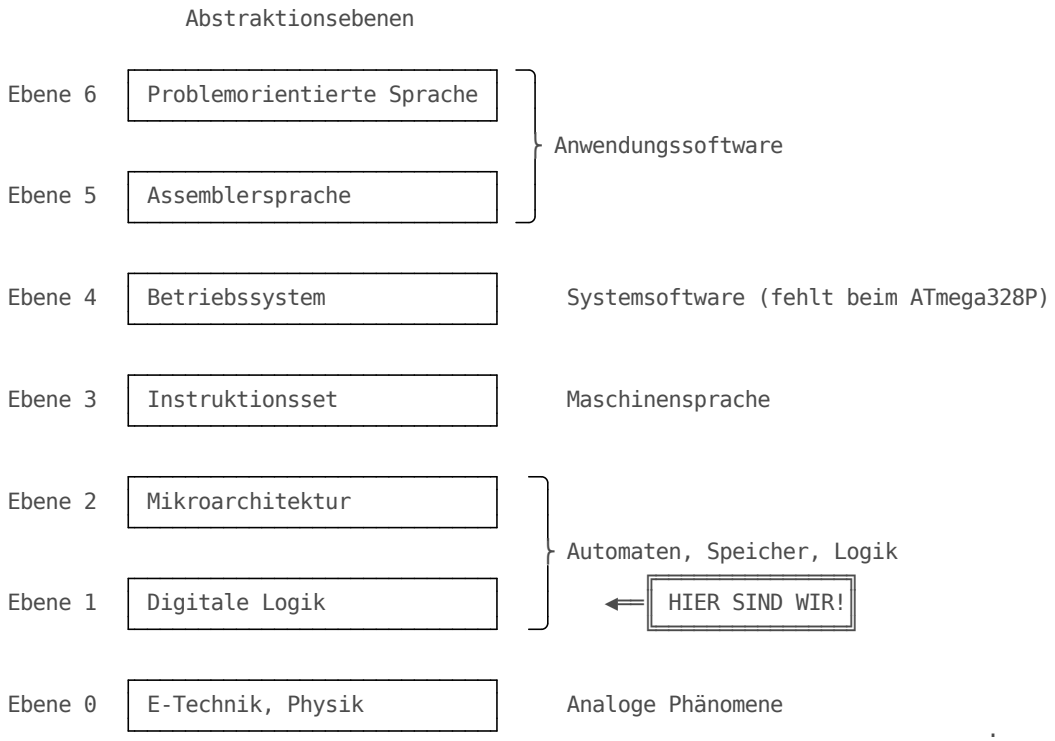
---

**Lernziele:** Nach dieser Vorlesung können Sie

- den Entwurfsweg für Schaltwerke an einem Moore-Automaten mit JK-Flip-Flops anwenden (Ampelsteuerung),
- erklären, wie ein Register aus D-Flip-Flops aufgebaut ist und wie eine Adresse aus mehreren Registern eines auswählt,
- die Ein- und Ausgabevarianten von Registern (SISO, SIPO, PISO, PIPO) unterscheiden,
- Zähler nach Taktung, Zahlenformat und Zählrichtung einordnen und einen synchronen Binärzähler mit D-Flip-Flops herleiten.

**🗺️ Diese Vorlesung in der Kurslandkarte →**

---




---

# Feedbacks/Ergänzungen

| Format | Informatik Studierende | Nicht-Informatik Studierende |
|---|---|---|
| Verbesserungsvorschlag | 5 | 2 |
| Fragen | 2 | 8 |
| generelle Hinweise | 0 | 1 |

In einer der Beispielaufgaben sollte ein Vergleicher entworfen werden, der zwei 2-Bit-Zahlen vergleicht und die Ausgänge für „gleich“, „größer als“ und „kleiner als“ bereitstellt.

Aus der Aufgabe ergäbe sich folgende Wahrheitstabelle:

| $a_1$**a1** | $a_0$**a0** | $b_1$**b1** | $b_0$**b0** | $Y_{A=B}$**YA=B** |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 1 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 0 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 1 | 1 | 1 |

Die Ausgänge können mit folgenden Gleichungen realisiert werden:

$$Y_{A=B} = \bar{a_1} \cdot \bar{a_0} \cdot \bar{b_1} \cdot \bar{b_0} + \bar{a_1} \cdot a_0 \cdot \bar{b_1} \cdot b_0 + a_1 \cdot \bar{a_0} \cdot b_1 \cdot \bar{b_0} + a_1 \cdot a_0 \cdot b_1 \cdot b_0$$

$$Y_{A>B} = \bar{a_1} \cdot a_0 \cdot \bar{b_1} \cdot \bar{b_0} + a_1 \cdot \bar{a_0} \cdot \bar{b_1} \cdot \bar{b_0} + a_1 \cdot \bar{a_0} \cdot \bar{b_1} \cdot b_0 + a_1 \cdot a_0 \cdot \bar{b_1} \cdot \bar{b_0} + a_1 \cdot a_0 \cdot \bar{b_1} \cdot b_0 + a_1 \cdot a_0 \cdot b_1 \cdot \bar{b_0}$$

$$Y_{A<B} = \bar{a_1} \cdot \bar{a_0} \cdot \bar{b_1} \cdot b_0 + \bar{a_1} \cdot \bar{a_0} \cdot b_1 \cdot \bar{b_0} + \bar{a_1} \cdot \bar{a_0} \cdot b_1 \cdot b_0 + \bar{a_1} \cdot a_0 \cdot b_1 \cdot \bar{b_0} + \bar{a_1} \cdot a_0 \cdot b_1 \cdot b_0 + a_1 \cdot \bar{a_0} \cdot b_1 \cdot b_0$$

YA=B YA>B YA<B = a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0 = a1 · a0 · b1

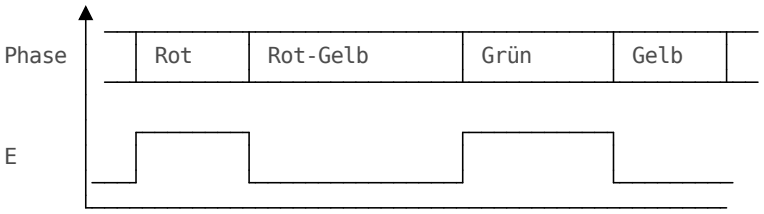
· b0 + a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0 = a1 · a0

· b1 · b0 + a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0 + a1 · a0 · b1 · b0

> Welche Vereinfachungsmöglichkeiten sehen Sie?

# Übungsbeispiel

Wir wollen eine Ampelanlage entwerfen, die durch einen Eingang $E$E getrieben wird, wobei der Wechsel zwischen den Zuständen von E eine Änderung der Ampelphase bewirkt. Die Umsetzung erfolgt als Moore-Automat.




.

### 1. Schritt: Aufstellen des Automatenmodels

```
PlantUML version 1.2026.9beta4 / 87fe16c [2026-09-25 19:18:22 UTC]
[From string (line 22) ]

@startuml
digraph finite_state_machine {
   rankdir=LR;

   node [shape = point ]; qi
   node[shape=circle]
   A[label="Rot"];
   B[label="Rot\nGelb"];
   C[label="Grün"];
   D[label="Gelb"];

   qi -> A;
   A  -> A  [ label = "E=1" ];
   A  -> B  [ label = "E=0" ];
   B  -> B  [ label = "E=0"];
   B  -> C  [ label = "E=1" ];
   C  -> C  [ label = "E= 1"];
   C  -> D  [ label = "E= 0"];
   D  -> D  [ label = "E= 0"];
   D  -> A [ label = "E= 1"];
}
@enduml
This looks like a DOT diagram. Please use @startdot instead of @startuml. (Assumed diagram type: sequence)
```

### 2. Schritt: Erstellen der Zustandstabelle

| Vorhergehender Zustand | Eingabe $E$E | Nachfolgezustand |
|---|---|---|
| Rot | 0 | RotGelb |
| Rot | 1 | Rot |
| RotGelb | 0 | RotGelb |
| RotGelb | 1 | Grün |
| Grün | 0 | Gelb |
| Grün | 1 | Grün |
| Gelb | 0 | Gelb |
| Gelb | 1 | Rot |

**Schritt 3: Auswahl einer binären Zustandskodierung und Generierung einer binären Zustandstabelle**

| Zustand | G | F |
|---|---|---|
| Rot | 0 | 0 |
| RotGelb | 0 | 1 |
| Grün | 1 | 0 |
| Gelb | 1 | 1 |

**Aufgabe:** Ergänzen Sie die Zustandsübergangstabelle!

| G | F | E | G' | F' |
|---|---|---|---|---|
| | | 0 | | |
| | | 1 | | |
| | | 0 | | |
| | | 1 | | |
| | | 0 | | |
| | | 1 | | |
| | | 0 | | |
| | | 1 | | |

Lösung:

| Vorhergehender Zustand | G | F | Eingabe $E$ E | Nachfolgezu |
|---|---|---|---|---|
| Rot | 0 | 0 | 0 | RotGelb |
| Rot | 0 | 0 | 1 | Rot |
| RotGelb | 0 | 1 | 0 | RotGelb |
| RotGelb | 0 | 1 | 1 | Grün |
| Grün | 1 | 0 | 0 | Gelb |
| Grün | 1 | 0 | 1 | Grün |
| Gelb | 1 | 1 | 0 | Gelb |
| Gelb | 1 | 1 | 1 | Rot |

**Schritt 4: Auswahl eines Flip-Flop Typs und Ermittlung der für jeden Zustandsübergang benötigten Flip-Flop Ansteuerungen**

Wir entscheiden uns für einen JK-Flip-Flop für die Realisierung. Die entsprechende invertierte Wahrheitstafel haben Sie zwischenzeitlich im Kopf:

| $Q(t)$**Q(t)** | $Q(t+1)$**Q(t + 1)** | $J$**J** | $K$**K** |
|---|---|---|---|
| 0 | 0 | 0 | $d$d |
| 0 | 1 | 1 | $d$d |
| 1 | 0 | $d$d | 1 |
| 1 | 1 | $d$d | 0 |

Schauen wir zunächst auf den Flip-Flop G, dessen Eingangsbelegung muss also auf die erwarteten Zustandsübergänge abgebildet werden.

| **G** | **F** | **E** | **G'** | **F'** |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 |

**Aufgabe:** Ergänzen Sie die Einträge für $JF$JF und $KF$KF!

| G | F | E | G' | F' |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 0 |

**Minimieren der Schaltung**

| G | F | E | JG | KG |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | d |
| 0 | 0 | 1 | 0 | d |
| 0 | 1 | 0 | 0 | d |
| 0 | 1 | 1 | 1 | d |
| 1 | 0 | 0 | d | 0 |
| 1 | 0 | 1 | d | 0 |
| 1 | 1 | 0 | d | 0 |
| 1 | 1 | 1 | d | 1 |

**Aufgabe:** Lesen Sie die minimierten Funktionen für die Flip-Flop-Eingänge ab!

JG

| | $\overline{GF}$ | $\overline{G}F$ | $GF$ | $G\overline{F}$ |
|---|---|---|---|---|
| $\overline{E}$ | | | d | d |
| $E$ | | 1 | d | d |

JF

| | $\overline{GF}$ | $\overline{G}F$ | $GF$ | $G\overline{F}$ |
|---|---|---|---|---|
| $\overline{E}$ | 1 | d | d | 1 |
| $E$ | | d | d | |

KG

| | $\overline{GF}$ | $\overline{G}F$ | $GF$ | $G\overline{F}$ |
|---|---|---|---|---|
| $\overline{E}$ | d | d | | |
| $E$ | d | d | 1 | |

KF

| | $\overline{GF}$ | $\overline{G}F$ | $GF$ | $G\overline{F}$ |
|---|---|---|---|---|
| $\overline{E}$ | d | | | d |
| $E$ | d | 1 | 1 | d |

.

Damit lassen sich folgende Funktionen ablesen:

$$\begin{aligned} JG &= FE \\ KG &= FE \\ JF &= \bar{E} \\ KF &= E \end{aligned}$$

JGKGJFKF = FE = FE = E = E

**Ausgabelogik**

Die Ansteuerung der drei Ausgabeleitungen R, Gr, Ge ergibt sich aus den Zuständen:

| **Zustand** | **G** | **F** | $R$**R** | $Gr$**Gr** |
|---|---|---|---|---|
| Rot | 0 | 0 | 11 | 00 |
| RotGelb | 0 | 1 | 11 | 00 |
| Grün | 1 | 0 | 00 | 11 |
| Gelb | 1 | 1 | 00 | 00 |

$$\begin{aligned} R &= \bar{G} \\ Gr &= G\bar{F} \\ Ge &= F \end{aligned}$$

RGrGe = G = GF = F

Wie sehen alternative Umsetzungen aus?



Simulation time: 00:37.860 (101%)

**statemachine.cpp**

```
typedef struct {
    int state;
    int next;
    int A_red;
    int A_yellow;
    int A_green;
    int timer;
} tl_state_t;    // Traffic light state

tl_state_t states[4] = {
// state     A_red             timer
//  |   next  |  A_yellow       |
//  |    |    |   |    A_green  |
//-----------------------------------------
{   0,   1,   1,  0,    0,      3},
{   1,   2,   1,  1,    0,      1 },
{   2,   3,   0,  0,    1,      3},
{   3,   0,   0,  1,    0,      1,}
};

const int greenPin = 11;
const int yellowPin = 12;
const int redPin = 13;
int state = 0;

void setup() {
  pinMode(greenPin, OUTPUT);
  pinMode(yellowPin, OUTPUT);
  pinMode(redPin, OUTPUT);
}

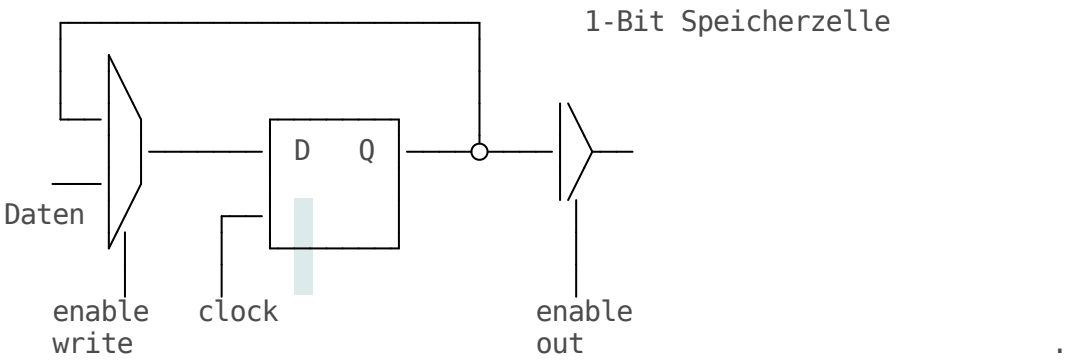
void loop() {
  if (states[state].A_red == 1) digitalWrite(redPin, HIGH);
  else digitalWrite(redPin, LOW);
  if (states[state].A_yellow == 1) digitalWrite(yellowPin, HIGH);
  else digitalWrite(yellowPin, LOW);
  if (states[state].A_green == 1) digitalWrite(greenPin, HIGH);
  else digitalWrite(greenPin, LOW);
  delay(states[state].timer*1000);
  state =  states[state].next;
}
```

```
Sketch uses 1150 bytes (3%) of program storage space. Maximum is 32256
bytes.
Global variables use 59 bytes (2%) of dynamic memory, leaving 1989
bytes for local variables. Maximum is 2048 bytes.
```

| Methode | Bauteilkosten | Entwurf | Variabilität |
|---|---|---|---|
| Analoge Schaltung | minimal | aufwändig | gering |
| Digitale Logik | gering | einfach | mittel |
| Software (eingebetteter $\mu$µC) | überschaubar | einfach | hoch |
| Software (PC) | hoch | sehr einfach | hoch |

# Register

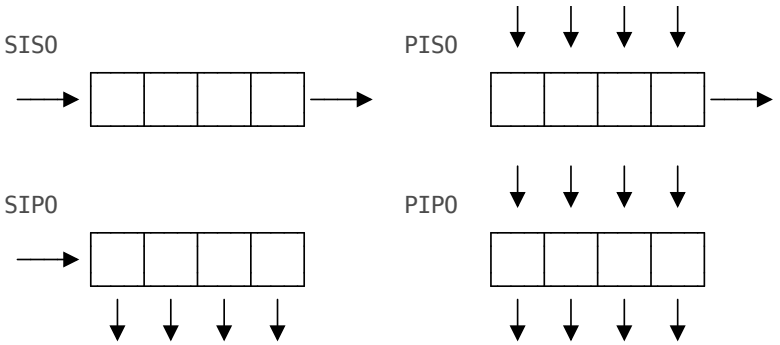
**Merke** Register bilden das Zusammenspiel mehrerer Flip-Flops.




.

Aggregieren wir nun mehrere D-Flip-Flops in einem Baustein, können wir wie nachfolgend gezeigt 4 Bit damit speichern. Der Flip-Flop $1Q$1Q könnte dabei die niedrigste Stelle in unser Zahlenrepresentation einnehmen, $Q4$Q4 die höchste. Um also $10111011$ in unserem Speicher abzulegen, müssten wir and den Inputs $D_1$D1 bis $D_4$D4 die zugehörigen Pinbelegungen realisieren, die dann mit dem nächsten Taktflankenwechsel übernommen werden. Die Übernahme der daten wird dabei durch zwei Enable Flags definiert, die $G, \bar{G}$G, G konfiguriert.

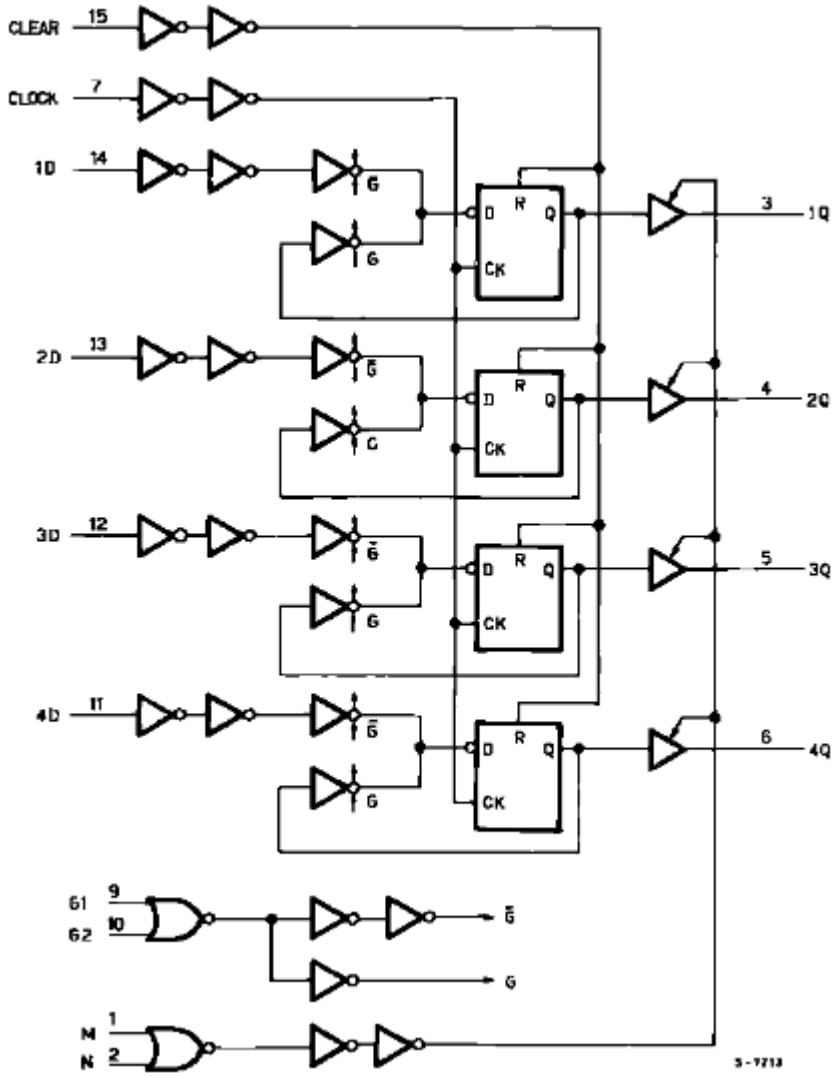
Die Eingabe der Daten kann seriell (serial input, SI) oder parallel erfolgen (parallel input, PI), ebenso die Ausgabe (serial output, SO bzw. parallel output, PO).



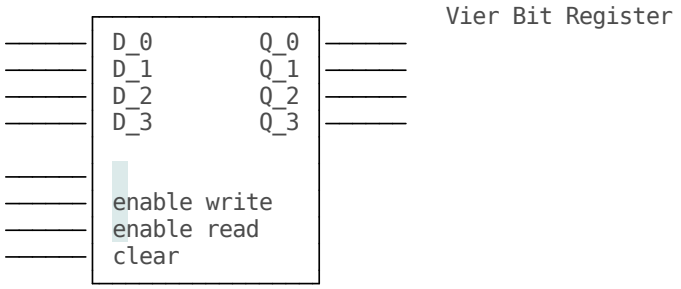

.

| **Prinzip** | **Anwendung** |
|---|---|
| SISO | FIFO-Speicher (first-in-first-out), Multiplikation bzw. Division mit dem Faktor 2 |
| SIPO | Umwandlung serielles Datenformat in paralleles Datenformat |
| PISO | Umwandlung paralleles Datenformat in serielles Datenformat |
| PIPO | Register zum reinen Zwischenspeichern, z. B. Akkumulator |

Schauen wir uns das Ganze mal am Beispiel eines 4 Bit PIPO Registers an:

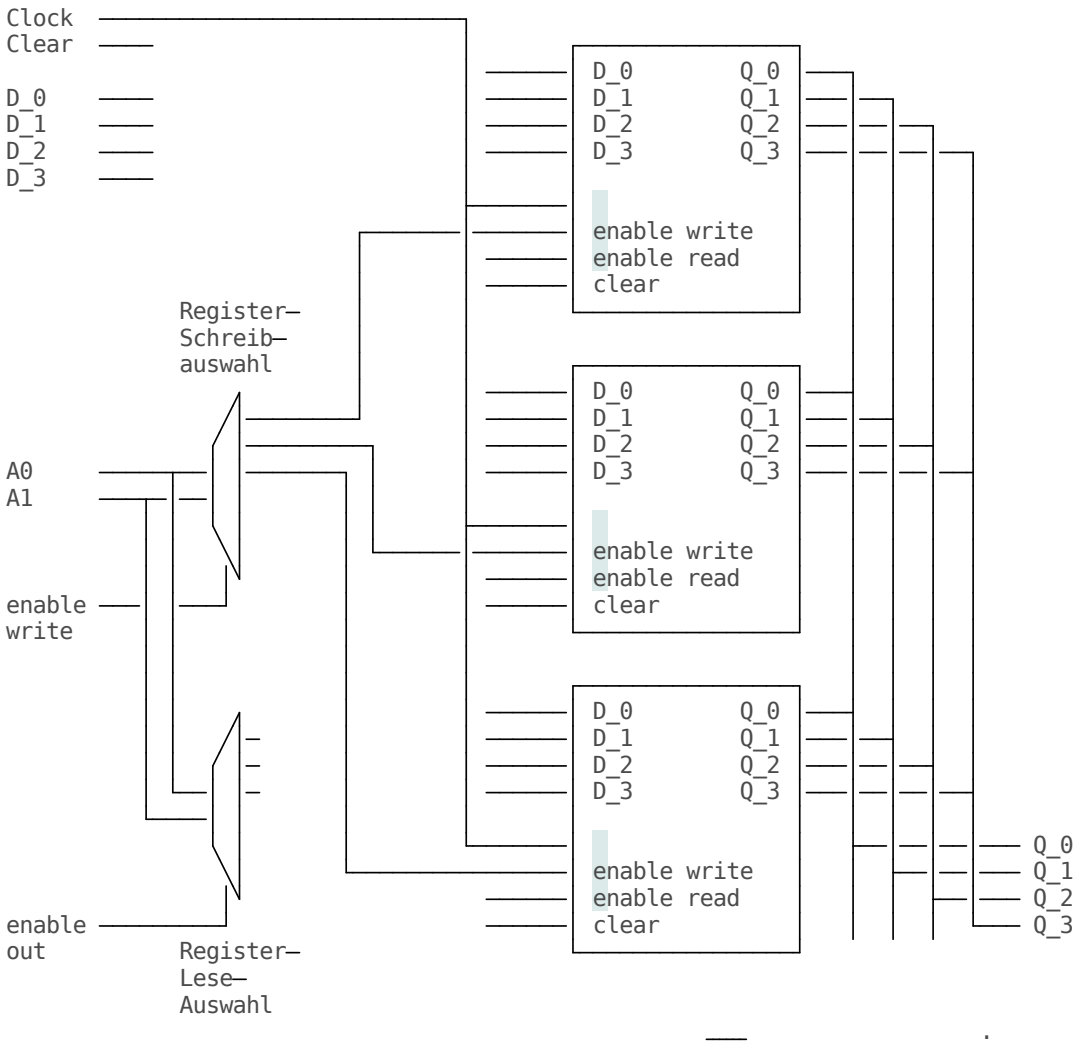



SGS Thomson, Datenblatt 74HC173, Quad D-Type Register [1]




In der Summe brauchen wir damit 4 Eingangsleitungen, 4 Ausgangsleitungen sowie diverse Steuerleitungen - für eine größere Anzahl an Registern eine zu aufwändige Realisierung - wir brauchen einen „Zuteilungsmechanismus“.

Entsprechend führen wir eine Adresse ein, diese kodiert, welches Register konkret angesprochen werden soll. Die Datenbreite bleibt erhalten.




An diesem Schaubild werden zwei zentrale Größen einer Rechnerarchitektur deutlich. Die Adressbusbreite und die Wortlänge bzw.. die Datenbusbreite des Systems z. B. 8, 16, 32 oder 64 Bit.

An dieser Stelle sei noch einmal auf die Erläuterungen zum Schieberegister aus der Veranstaltung L6 verwiesen. Dieses erweitert die Funktionalität und ermöglicht neben dem Abspeichern und Lesen auch Shiftoperationen.

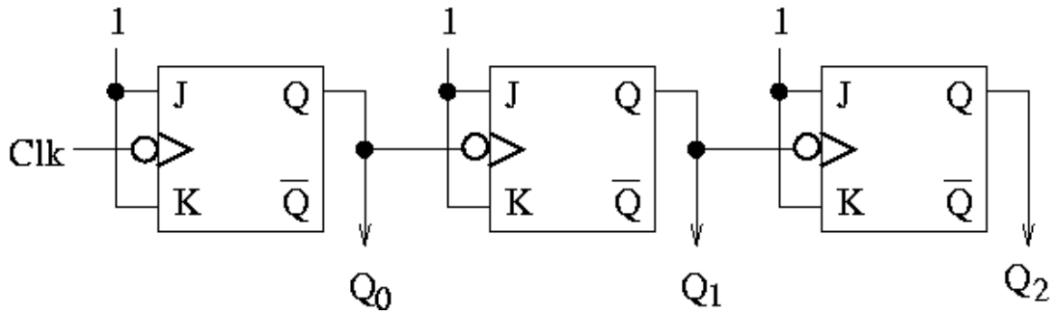
---

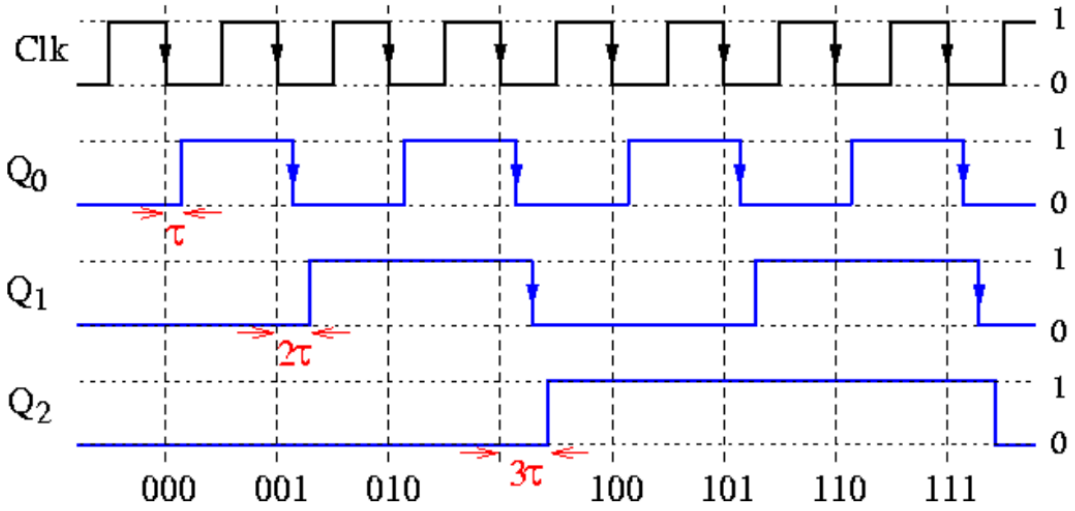
[1] SGS Thomson, Datenblatt 74HC173, „Quad D-Type Register“, Link

# Zähler

Eine sequentielle Schaltung, die Eingangsimpulse auf Zustände abbildet nennt man Zähler. Mit ihnen lassen sich zeitlich aufeinander folgende Ereignisse registrieren aber auch erzeugen.

- **Synchron vs. asynchron** In einem synchronen Zähler sind alle Zähler-Speicher durch eine Taktleitung getriggert. Der Zustandswechsel wird immer zugleich vollzogen. Asynchrone Zähler heben diese Trennung auf und erlauben eine vereinfachte Umsetzung. Allerdings ist die Gültigkeit der Ergebnisse dann von dem Laufzeitverhalten der Komponenten abhängig.




- **Zahlenformat** Die binäre Zahlenrepräsentation ist die intuitive Darstellung des Zahlenwertes. Alternativ lassen sich aber auch Dezimale Zähler, beruhend auf z. B. dem BCD-Code oder Zähler mit anderen Codes arbeiten, z. B. Gray-Code umsetzen.
- **Zählrichtung** Unidirektionale Zähler können ausschließlich vorwärts oder rückwärts zählen (inkrementieren/dekrementieren). Bidirektionale Zähler unterstützen beide Zählrichtungen, benötigen aber auch eine separate Steuerleitung.

Leiten wir uns einen synchronen 4 Bit Zähler zur Vertiefung des Verständnisses noch mal her:

```
PlantUML version 1.2026.9beta4 / 87fe16c [2026-09-25 19:18:22 UTC]
[From string (line 22) ]

@startuml
digraph finite_state_machine {
   rankdir=LR;

   node [shape = point ]; qi
   node[shape=circle]
   A[label="0000"];
   B[label="0001"];
   C[label="..."];
   D[label="1111"];

   qi -> A;
   A -> A [ label = "E=0" ];
   A -> B [ label = "E=1" ];
   B -> B [ label = "E=0"];
   B -> C [ label = "E=1" ];
   C -> C [ label = "E= 0"];
   C -> D [ label = "E= 1"];
   D -> D [ label = "E= 0"];
   D -> A [ label = "E= 1"];
}
@enduml
This looks like a DOT diagram. Please use @startdot instead of @startuml. (Assumed diagram type: sequence)
```




Offenbar sind 4 Zustände notwendig, um die Zahlen von 0-15 abbilden zu können. Die Zustandsübergangstabelle ergibt sich folglich zu.

| F | G | H | I | F' |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 0 |
| 0 | 0 | 1 | 1 | 0 |
| 0 | 1 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 |
| 0 | 1 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 |
| 1 | 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 1 | 1 |
| 1 | 1 | 0 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 |
| 1 | 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 | 0 |

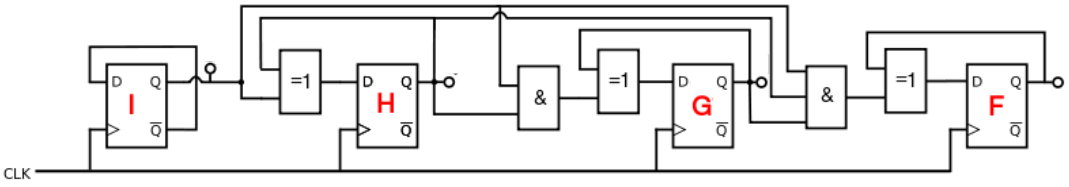
Unter der Vorgabe, dass wir D-Flip-Flops für die Umsetzung verwenden, sind die Eingänge der Speicher mit den Zuständen F' bis I' zu beschalten.

In der Vereinfachung ergeben sich daraus folgende Gleichungen:

$$
\begin{aligned}
DF &= \bar{I}F + \bar{G}F + \bar{H}F + IHG\bar{F} \\
DG &= \bar{I}G + \bar{H}G + IH\bar{G} \\
DH &= I\bar{H} + \bar{I}H \\
DI &= \bar{I}
\end{aligned}
$$

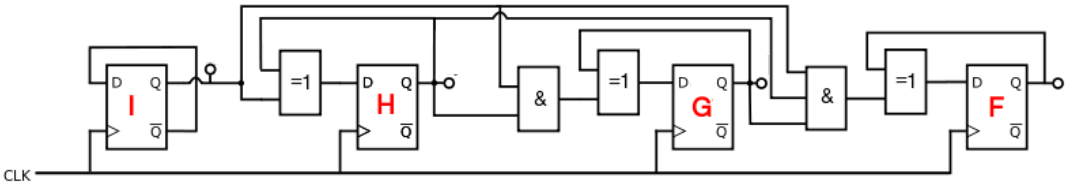
DF DG DH DI = IF + GF + HF + IHGF = IG + HG + IHG = IH + IH = I

Im Ergbnis lässt sich das System mit entsprechenden XOR und AND Bausteinen Umsetzen.




**Aufgabe:** Leiten Sie die Im Schaltwerk gezeigte Realisierung aus den oben genannten Gleichungen her.

**Aufgabe:** Leiten Sie die Im Schaltwerk gezeigte Realisierung aus den oben genannten Gleichungen her.




Für $DI$DI und $DH$DH ist die Lösung trivial, für DG müssen wir den Term etwas umstellen. Die Schaltung drückt die Gleichung $DG = IH \oplus G$DG = IH ⊕ G aus.

$$
\begin{aligned}
DG &= IH \oplus G \\
&= \bar{I}HG + IH\bar{G} \\
&= (\bar{I} + \bar{H})G + IH\bar{G} \\
DG &= \bar{I}G + \bar{H}G + IH\bar{G}
\end{aligned}
$$

DGDG = IH ⊕ G = IHG + IHG = (I + H)G + IHG = IG + HG + IHG

**Merke** Aus der Kombination der genannten Klassifkationsmerkmale für Zähler lassen sich spezifische Umsetzungen realisieren.

Entwurf synchroner Zählerschaltungen
Christian Stöcker

Ansehen auf

**Alternative Implementierung mit erweiterter Funktionalität:**

Der Zustand eines Zählerbits **wechselt**, wenn das nächst kleinere Bit **von 1 zu 0** springen wird.

| a | b | c | d | a' | b' | c' | d' |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| 0 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| 0 | 0 | 1 | 1 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 1 | 0 | 1 | 1 | 0 |
| 0 | 1 | 1 | 0 | 0 | 1 | 1 | 1 |
| 0 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| 1 | 0 | 0 | 1 | 1 | 0 | 1 | 0 |
| 1 | 0 | 1 | 0 | 1 | 0 | 1 | 1 |
| 1 | 0 | 1 | 1 | 1 | 1 | 0 | 0 |
| 1 | 1 | 0 | 0 | 1 | 1 | 0 | 1 |
| 1 | 1 | 0 | 1 | 1 | 1 | 1 | 0 |
| 1 | 1 | 1 | 0 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |

Das bedingte Wechseln eines Bits lässt sich mit XOR realisieren:

$$\begin{aligned} x \oplus 0 &= x \\ x \oplus 1 &= \bar{x} \end{aligned}$$

x ⊕ 0x ⊕ 1 = x = x

Wenn man möchte, dass $x$x wechselt, wenn $x_{flip}$xflip high ist, dann kann $x \oplus x_{flip}$x ⊕ xflip verwendet werden.

Aus der Bedingung oben ergibt sich:

$$\begin{aligned} a_{flip} &= b_{flip} \cdot b \\ a &= a \oplus a_{flip} \end{aligned}$$

aflipa = bflip · b = a ⊕ aflip

$a$a wird wechseln, wenn $b$b wechseln wird und $b = 1$b = 1 gilt, wobei b das nächst niederwertigste Bit von a ist. $b_{flip}$bflip lässt sich wie $a_{flip}$aflip berechnen, wenn es nicht das niederwertigste Bit ist. Das niederwertigste Bit wechselt dann wenn der Eingang $count$count (carryIn) auf High ist.

Der Ausgang $carryOut$carryOut lässt sich berechnen mit $a_{flip} \cdot a$aflip · a.

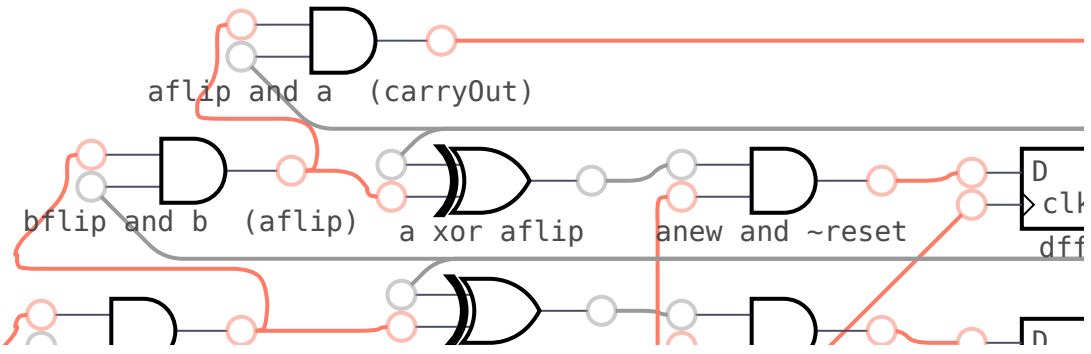
Für einen 4 Bit Zähler mit den Bits a,b,c,d (a höchstwertig, d niederwertigste) ergeben sich diese Formeln:

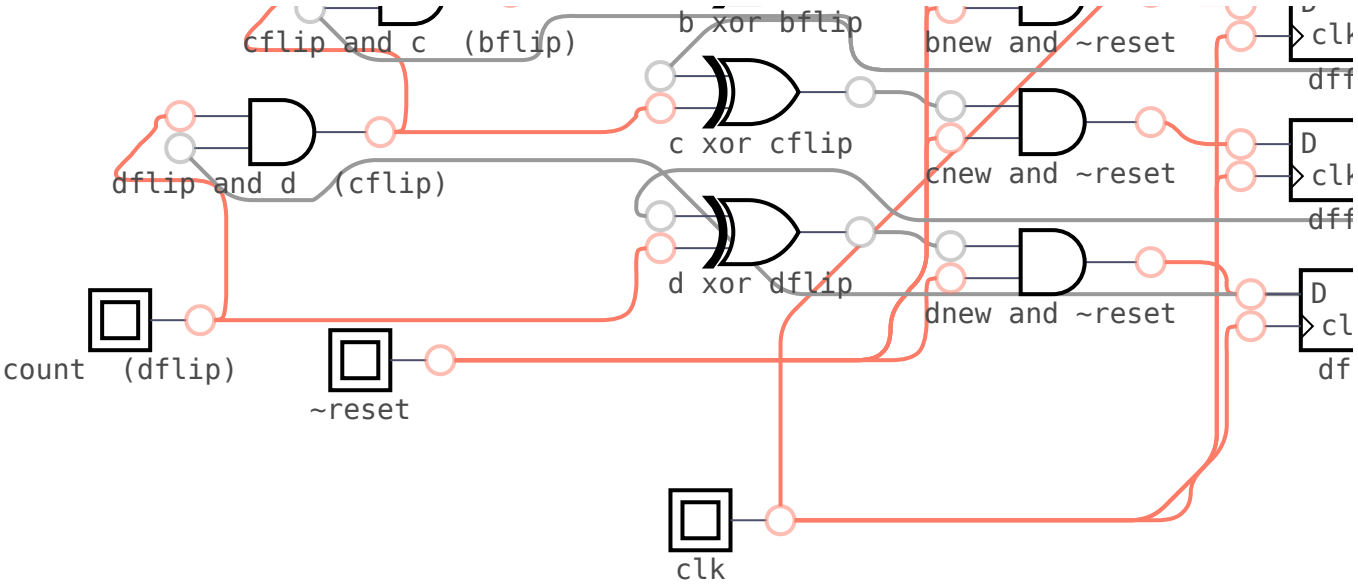
$$\begin{aligned} d_{flip} &= count \\ d^{'} &= d_{flip} \oplus d \\ c_{flip} &= d_{flip} \cdot d \\ c^{'} &= c \oplus c_{flip} \\ b_{flip} &= c_{flip} \cdot c \\ b^{'} &= b \oplus b_{flip} \\ a_{flip} &= b_{flip} \cdot b \\ a^{'} &= a \oplus a_{flip} \\ carryOut &= a_{flip} \cdot a \end{aligned}$$

dflipd′cflipc′bflipb′aflipa′carryOut = count = dflip ⊕ d = dflip · d = c ⊕ cflip = cflip · c = b ⊕ bflip = bflip

· b = a ⊕ aflip = aflip · a

Aus technischen Gründen hat die folgende Simulation noch eine Reset Schaltung (jeder neue Zustand wird noch mit $r\bar{e}set$reset UND genommen.) Am Anfang `count` und `~reset` auf LOW lassen, clock puls geben und dann `count` und `~reset` auf HIGH setzen um bei dem nächten clock Puls das Zählen zu beginnen.






Nachteil dieser Schaltung: höhere Laufzeiten

## Was Sie heute mitnehmen

Diese fünf Punkte fassen die Vorlesung zusammen. Register und Zähler kehren in jeder Rechnerarchitektur wieder; prüfen Sie mit den Fragen darunter, ob Sie ihren Aufbau erklären und einen Zähler selbst herleiten können.

- **Der Entwurfsweg trägt auch größere Aufgaben:** Die Ampelsteuerung entsteht als Moore-Automat mit vier Zuständen, zwei JK-Flip-Flops und einer Ausgabelogik, die aus dem Zustand die drei Lampen ansteuert.
- **Register** fassen mehrere D-Flip-Flops zusammen, die gemeinsam ein Datenwort speichern. Daten können seriell oder parallel ein- und ausgegeben werden (SISO, SIPO, PISO, PIPO); ein **Schieberegister** kann seinen Inhalt zusätzlich verschieben.
- **Adressierung:** Bei vielen Registern wählt eine Adresse aus, welches Register beschrieben oder gelesen wird. Dabei werden **Adressbusbreite** und **Datenbusbreite** als zentrale Größen einer Rechnerarchitektur sichtbar.
- **Zähler** bilden Eingangsimpulse auf Zustände ab. Man unterscheidet sie nach Taktung (synchron oder asynchron), Zahlenformat (binär, BCD, Gray-Code) und Zählrichtung; asynchrone Zähler sind einfacher, ihr Ergebnis hängt aber von den Laufzeiten der Bauteile ab.
- **Synchroner Binärzähler:** Ein Zählerbit wechselt, wenn das nächst niedrigere Bit von 1 auf 0 springt. Mit D-Flip-Flops ergibt das eine Schaltung aus XOR- und AND-Gattern.

**Prüfen Sie sich:**

- Welche Funktionalität bieten Standard-Schaltwerke wie Zähler und Register?
- Wie leiten Sie für die Ampelsteuerung aus der Zustandstabelle die Beschaltung der JK-Flip-Flops und die Ausgabelogik ab?
- Wie ist ein 4-Bit-Register aus D-Flip-Flops aufgebaut, und warum führt man bei vielen Registern eine Adresse ein?
- Was ist ein Schieberegister, und welche Ein- und Ausgabevarianten (SISO, SIPO, PISO, PIPO) gibt es?
- Wie unterscheiden sich synchrone und asynchrone Zähler, und welchen Nachteil erkauft man sich mit der einfacheren asynchronen Umsetzung?
- Wie viele Flip-Flops braucht ein synchroner 4-Bit-Binärzähler, und warum lässt sich jedes Zählerbit mit einem XOR-Gatter umschalten?

## Ihre Begriffe für die Kurslandkarte

Die Kurslandkarte wächst mit dem Semester, und Sie bestimmen mit, was hineinkommt. Ihre Antworten sammeln wir im Hörsaal gemeinsam über das LiaScript-Klassenzimmer; nach der Vorlesung werden die Begriffe dieser Woche in der Karte freigeschaltet.

**Welche Begriffe aus der heutigen Vorlesung gehören für Sie in die Kurslandkarte?**

Texteingabe ...

**Wo begegnet Ihnen das, was heute dran war – in Ihrem Studiengang oder in Geräten Ihres Alltags?**

Texteingabe ...

## Übungsaufgaben

- Entwerfen Sie einen Zähler, der eine Zählrichtungsvorgabe ermöglicht und evaluieren Sie Ihre Lösung mit einem Simulator.

  logisim-evolution

- Recherchieren Sie die Umsetzung eines Johnson Zählers und übernehmen Sie eine Implementierung in eine Simulator.